

BookRAG

Agentic, Grounded Retrieval-Augmented Generation
over a Multimodal Finance Corpus

Technical Design & Engineering Documentation

*A production-minded RAG system whose guiding principle is **zero hallucination**: every answer is grounded in retrieved sources, verified by a strict fact-checking gate, and cited — with the pipeline visualized live, node by node.*

Contents

1. Executive Summary
 2. Motivation & Design Goals
 3. System Architecture
 4. Data Acquisition (legal, automated)
 5. Multimodal Ingestion
 6. Hybrid Retrieval & Rank Fusion
 7. The Agentic Pipeline (LangGraph)
 8. Backend & Real-Time Streaming
 9. Frontend & User Experience
 10. Persistence & Long-Term Memory
 11. Configuration Reference
 12. Running the System
 13. Engineering Challenges & Solutions
 14. Design Decisions & Trade-offs
 15. Roadmap
- Appendix: Tech Stack & Repository Layout

1. Executive Summary

BookRAG is an end-to-end Retrieval-Augmented Generation (RAG) system that answers natural-language questions about finance, business and wealth using a curated corpus of ~206 documents and ~40,000 indexed passages. Unlike a typical chatbot, it is engineered around a single non-negotiable goal: **grounded answers with zero hallucination**. Every response must be traceable to retrieved source excerpts, is checked by an independent fact-verification step, and is cited back to the source document.

Three properties distinguish it from a standard RAG demo. First, the corpus is **genuinely multimodal** — SEC filings dense with financial tables, glossy annual-report PDFs, scanned public-domain texts and quantitative-finance papers — all acquired automatically from free, legal, API-driven sources. Second, the retrieval and generation loop is **agentic**: it rewrites queries, fuses two retrieval strategies, verifies grounding, and automatically retries with an expanded query when an answer cannot be verified. Third, the normally-invisible pipeline is rendered as a **live, interactive visualization**, so any observer can see exactly what the system is doing in real time, with per-node timings, without reading logs.

The entire system runs **free and without a GPU**: cloud inference via Groq's free tier for generation, and local CPU / Apple-Silicon (MPS) embeddings.

2. Motivation & Design Goals

Most RAG tutorials index a handful of clean text files and call a model. That hides the hard, real-world problems: messy multimodal documents, retrieval that misses, models that confidently invent facts, and pipelines that are opaque black boxes. BookRAG was built to confront each of these.

- **Correctness over latency.** Quality is the priority. The pipeline runs up to two full generate-then-verify cycles rather than returning a fast but ungrounded answer.
- **Multimodal from the start.** Financial value lives in tables and filings, not just prose, so ingestion must extract and preserve tabular data, not discard it.
- **Legally clean & reproducible.** Every data source is public-domain, open, or government data, so the project is redistributable and portfolio-safe.
- **Observable.** The pipeline is a first-class UI citizen — developers, reviewers and stakeholders can watch and inspect every stage.
- **Accessible.** It must run on a commodity laptop with no GPU and no paid API.

3. System Architecture

BookRAG has three stages: an offline **acquisition** stage that builds the corpus, an offline **ingestion** stage that turns documents into a searchable index, and an online **query** stage where a FastAPI backend runs the agentic pipeline and streams every step to a Next.js UI.

```
██████████ acquire.py ██████████
■ SEC EDGAR · World Bank · arXiv q-fin ■ ███ corpus/ + manifest.json
■ annualreports.com · Internet Archive ■
████████████████████████████████████████████████████████████████████████████████
██████████
██████████ ingest.py ██████████
■ parse (text + tables + OCR / HTML) ■ ███ chroma_db/
■ → chunk → BGE embed → Chroma + BM25 ■ (dense vectors + sparse index)
```



Figure 1 — Three-stage architecture: acquire → ingest → serve.

The clean separation means the expensive offline stages run once, while the online stage stays lightweight and streams results incrementally for a responsive experience.

4. Data Acquisition (legal, automated)

The `acquire.py` CLI and its per-source downloader modules build the corpus from official APIs. The acquisition is idempotent and resumable: a JSON manifest records every document by URL and content hash, so runs can be re-run to top up toward a target count, and duplicates (by URL or bytes) are never stored twice. Each source is polite — rate-limited with an honest User-Agent.

Source	Content	Format	Why it matters
SEC EDGAR	10-K / 20-F filings	HTML	Dense financial statement tables
World Bank	Open economic reports	PDF	Charts, data tables, infographics
arXiv q-fin	Quant-finance papers	PDF	Equations, plots, results tables
Internet Archive	Public-domain finance texts	PDF	Scanned image pages (OCR path)
annualreports.com	Corporate annual reports	PDF	Designed, chart-heavy layouts

Table 1 — Corpus sources. All free / open / government data.

5. Multimodal Ingestion

Ingestion (`ingest.py` + `utils/parsing.py`) converts every document into a flat list of provenance-tagged elements, then into embedded, indexed chunks. The parser routes by file type and, crucially, treats tables as first-class content.

Parsing

- **PDF text** is extracted per page with PyMuPDF. Pages with almost no text are treated as scanned and sent to an optional OCR fallback (auto-detected).
- **PDF tables** are detected natively and serialized to Markdown, so numbers survive into the index.
- **HTML SEC filings** are XHTML, so each `<table>` tag is parsed individually with `lxml`, cleaned of colspan spacer artifacts, filtered to number-dense data tables, and rendered as Markdown; the narrative text is extracted separately so figures are not duplicated as prose.

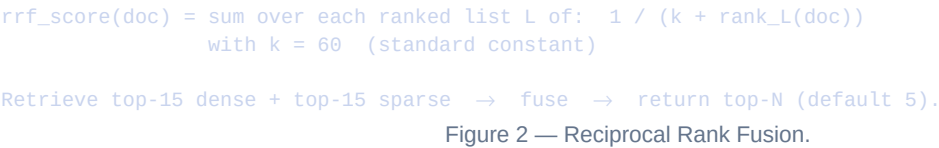
Chunking, embedding & indexing

Text elements are split with a recursive, paragraph-first splitter (chunk size 800, overlap 150). Tables are kept whole when they fit, else split by rows with the header repeated on each piece so a financial table

never loses its column labels. Every chunk carries flat metadata (source, title, page, element type). Chunks are embedded with BAAI/bge-small-en-v1.5 — using the correct asymmetric convention (passages unprefixed, queries prefixed), the single most common cause of poor BGE retrieval — and written to a persistent Chroma collection (cosine) plus a BM25 index. A per-document chunk cap (default 250, evenly strided) bounds memory and index size so the whole corpus fits comfortably in 8 GB of RAM.

6. Hybrid Retrieval & Rank Fusion

Dense vector search excels at semantic similarity; sparse keyword search (BM25) excels at exact terms (tickers, statute names, numbers). BookRAG runs both and fuses them with Reciprocal Rank Fusion (RRF), which combines rankings without needing to normalize incompatible score scales.



On a retry, the first attempt's documents are merged into the candidate pool before re-fusing, so the second attempt always has **more** context, never less — a key anti-hallucination strategy.

7. The Agentic Pipeline (LangGraph)

The pipeline is a LangGraph state machine. A lightweight model handles routing and expansion; a stronger model handles generation and grounding evaluation. Structured outputs use plain-text JSON with defensive parsing rather than tool-calling, for maximum endpoint compatibility.

Node	Role	What it does
rewrite_and_route	router	Rewrites the query; routes to general vs. RAG.
retrieve	—	Dense + sparse search, RRF fusion, retry-merge.
generate	heavy	Answers using only excerpts (text + tables); cites sources.
evaluate_grounding	heavy	Strict fact-check: is every claim traceable? Else reject.
expand_query	router	On failure (once): expand query, retry retrieval.
general_answer	general	Small talk / out-of-scope, with a domain nudge.
build_final_answer	—	Assembles answer + citations + disclaimer if unverified.

Table 2 — Pipeline nodes.

The grounding gate is the heart of the system. It is deliberately strict: any claim not directly supported by an excerpt marks the answer ungrounded, triggering exactly one expand-and-retry cycle. If the answer still cannot be verified, it is returned with an honest caution disclaimer rather than presented as fact.

8. Backend & Real-Time Streaming

The FastAPI backend (server/) exposes chat CRUD endpoints and a streaming endpoint that runs the pipeline and emits Server-Sent Events. As LangGraph executes, the backend infers node transitions (mirroring the graph's edges) and emits a live event stream:

```

node_start { node }
node_end   { node, dur_ms, summary }      ← per-node timing + human summary
sources    { [ {title, source, page, element_type, rrf_score} ] }
trace      { route, rewritten_query, is_grounded, retry_count, ... }
token      { text }                      ← answer streamed for a typing effect
done | error

```

Figure 3 — The SSE event protocol driving the live UI.

9. Frontend & User Experience

The Next.js 15 frontend renders three synchronized panels. The result is that the pipeline — usually a black box — becomes something you can watch and inspect.

- **Left — live pipeline flowchart** (React Flow): nodes light up and show timings as they execute; an auto-following dialogue box explains the active node; fully interactive (drag, zoom, click).
- **Center — conversation**: streamed Markdown answers with source-citation cards, a grounding verdict badge, and a progress bar synced to real pipeline progress.
- **Right — trace timeline**: every executed node with a proportional duration bar and summary; click any node to inspect it (selection is bidirectional with the flowchart).

The app supports light/dark themes, panel minimize/maximize, and persistent multi-chat sessions.

10. Persistence & Long-Term Memory

Chats, messages, and a cross-chat user profile are persisted in SQLite. Two bounded layers of long-term context keep prompts small while making the assistant progressively more personalized:

- **Rolling summary** — once un-summarized turns accumulate, the oldest are compressed into a running per-chat summary, so context stays bounded no matter how long a conversation runs.
- **User profile snapshotting** — periodically, durable facts about the user (interests, goals, expertise, risk appetite) are distilled into a profile that carries across chats and personalizes future answers. These updates run after a response streams, so they add no perceived latency.

11. Configuration Reference

Everything is configured in `config.py` and overridable via environment variables. The most useful:

Variable	Default	Purpose
GROQ_API_KEY	—	Free Groq key; if set, provider auto-selects Groq.
BOOKRAG_MODEL_HEAVY	llama-3.1-8b-instant	Generation/grounding model (70B for higher quality).
BOOKRAG MOCK_LLM	0	1 = run the pipeline with a fake model (no key).
BOOKRAG_N_FINAL	5	Excerpts sent to the LLM (higher = richer, more tokens).
BOOKRAG_MAX_CHUNKS_PER_DOC	250	Caps chunks/doc; bounds RAM & index size.
BOOKRAG_EMBED_DEVICE	auto	auto uses Apple-Silicon GPU (MPS) if available.

12. Running the System

```
# 1. install (creates .venv, installs Python + frontend deps)
./setup.sh

# 2. add your free Groq key
cp .env.example .env      # set GROQ_API_KEY=gsk...

# 3. build corpus + index (one-time)
. .venv/bin/activate
export CONTACT_EMAIL="you@example.com"
python acquire.py          # ~200 legal multimodal docs → corpus/
python ingest.py           # parse → chunk → embed → index → chroma_db/

# 4. run (backend :8000 + UI :5200)
./run.sh                  # then open http://localhost:5200

# try the UI with no key / no corpus:
BOOKRAG MOCK_LLM=1 ./run.sh
```

13. Engineering Challenges & Solutions

A selection of real problems solved during development — the kind that never appear in tutorials.

- **SEC financial tables were being dropped**

Filings are XHTML, so a whole-document table parse silently failed (missing an optional dependency). Fixed by parsing each table element individually with lxml, then cleaning colspan spacer columns and filtering to number-dense data tables. Result: ~90 clean financial tables per filing.

- **Ingestion swap-thrashed an 8 GB machine**

The full corpus produced 138k chunks; holding them plus a 138k-vector in-memory index blew past RAM into 8 GB of swap and wedged the process. Fixed with a per-document chunk cap (evenly strided to keep coverage), Apple-Silicon (MPS) embeddings, and freeing intermediate objects — cutting to ~40k chunks that fit comfortably and index in minutes.

- **The live UI received zero streaming events**

The frontend split Server-Sent Events on a newline delimiter, but the SSE library frames events with carriage-return-newline pairs, so no event ever parsed. A one-line delimiter fix restored the entire live pipeline. A separate dev-proxy buffering issue was resolved by connecting the browser directly to the backend.

- **Groq free-tier rate limit (429)**

Each RAG turn called the 70B model twice with large table-heavy prompts, exhausting the 100k tokens/day cap. Fixed by defaulting to the 8B model (a far larger free budget), trimming to 5 excerpts, and capping each excerpt's length — roughly a 10x increase in daily capacity.

- **Empty flowchart & stats after launch**

Metadata was fetched once on page load; if the backend was still warming up (loading the index + embedding model), the fetch failed and never retried. Fixed with automatic retry until the backend is ready, so the UI self-heals.

14. Design Decisions & Trade-offs

- **Hybrid over pure-vector retrieval.** BM25 recovers exact-match terms (tickers, figures) that dense embeddings blur; RRF fuses them without score normalization.
- **Strict grounding + honest refusal.** The system prefers saying ‘I can’t verify this’ to guessing — the correct default for a finance assistant.
- **Provider-agnostic LLM layer.** Any OpenAI-compatible endpoint works (Groq, OpenAI, local Ollama), so cost, speed and privacy are configuration, not code.
- **Node-level streaming.** Streaming pipeline stages (not just tokens) makes the system observable and gives users honest progress feedback.
- **Per-document chunk cap.** A pragmatic bound that trades exhaustive coverage of a few giant filings for a system that runs on commodity hardware and retrieves with a better signal-to-noise ratio.

15. Roadmap

- Deployed public demo (managed frontend + hosted backend).
- Approximate / quantized index for faster retrieval at larger scale.
- Layout-aware SEC table normalization (e.g. docling).
- Per-token LLM streaming end-to-end.
- An evaluation harness measuring retrieval hit-rate and grounding accuracy.

Appendix: Tech Stack & Repository Layout

Layer	Technology
Orchestration	LangGraph
LLM	Groq (llama-3.1-8b-instant), OpenAI-compatible — swappable
Embeddings	BAAI/bge-small-en-v1.5 (CPU / Apple-Silicon MPS)
Vector store	ChromaDB (cosine) + rank-bm25 + RRF
Parsing	PyMuPDF, pdfplumber, BeautifulSoup + pandas
Backend	FastAPI + SSE, SQLite
Frontend	Next.js 15, React Flow, Tailwind CSS v4, Framer Motion

```

bookrag/
├── acquire.py      corpus acquisition CLI
├── ingest.py       multimodal ingestion pipeline
├── pipeline.py     LangGraph agentic RAG pipeline
├── config.py       central configuration
├── downloaders/   one module per data source
├── utils/         parsing, chunking, embeddings, retrieval, history, llm
├── server/        FastAPI backend: SSE streaming, SQLite chats, memory
└── webnext/       Next.js frontend (the live 3-panel UI)

```

BookRAG — an end-to-end demonstration of production-minded, grounded RAG.